

Informe Tècnic : Password Manager

Descripció del projecte

Password Manager és una aplicació web orientada a la generació, anàlisi i emmagatzematge segur de contrasenyes. El projecte neix de la necessitat actual de gestionar credencials sense reutilitzar contrasenyes febles ni comprometre la privacitat de l'usuari.

L'objectiu principal és combinar desenvolupament web modern, ciberseguretat i intel·ligència artificial dins una eina pràctica i funcional.

La característica més important del sistema és el seu enfocament Zero-Knowledge ,es a dir les contrasenyes es processen i es xifren localment abans de ser enviades al núvol, en aquest cas a Firebase Realtime Database , evitant emmagatzemar dades delicades en text pla.

Funcionalitats principals:

- Generació segura de contrasenyes.
- Anàlisi de seguretat.
- Recomanacions intel·ligents mitjançant IA.
- Emmagatzematge xifrat al núvol.
- Historial de contrasenyes.
- Dashboard amb estadístiques de seguretat.

URL del repositori

El codi font complet, el control de versions i l'historial de commits es troben disponibles a:

[GitHub - allobera1341-a11y/PasswordManager · GitHub](https://github.com/allobera1341-a11y/PasswordManager)

Ús de la IA. Integració i finalitat

La intel·ligència artificial s'ha utilitzat amb una doble funció, s'ha utilitzat com a eina de desenvolupament i com a servei integrat dins l'aplicació.

IA com a Assistent de Desenvolupament

S'han fet servir models de llenguatge avançats per accelerar el disseny de l'arquitectura de components en React, optimitzar les consultes a Firebase i aplicar estils de Tailwind CSS de forma eficient.

IA integrada a l'Aplicació

L'aplicació utilitza models mitjançant l'API de Google Gemini i OpenAI, utilitzant també ChatAnywhere per la gestió de claus. La seva finalitat és actuar com un assessor de Seguretat. L'arquitectura filtra les peticions per enviar únicament metadades com la longitud i puntuació, d'aquesta manera, l'API de la IA ofereix consells sense tenir accés en cap moment a la contrasenya real en text pla.

Fluxos de treball

El projecte s'ha executat seguint una metodologia dividida en 4 fases en un període de 7 hores.

En una primera fase es va definir l'estructura general del projecte, la distribució de carpetes i els components principals de la interfície amb React i Vite. També es va començar el disseny visual utilitzant Tailwind CSS per crear una interfície moderna basada en targetes, dashboards i components reutilitzables.

Posteriorment, es va implementar la part de seguretat local de l'aplicació. Durant aquesta fase es van desenvolupar els mòduls de generació de contrasenyes, l'anàlisi de seguretat i el sistema de xifrat local mitjançant Web Crypto API i AES-256-GCM. L'objectiu era garantir que les dades sensibles mai sortissin del navegador en text pla.

A continuació es va integrar Firebase Authentication i Firebase Realtime Database per gestionar la persistència de dades i l'autenticació dels usuaris. També es va implementar la sincronització de l'historial de contrasenyes segons l'identificador únic de cada usuari autenticat.

Finalment, es va treballar en la integració de la intel·ligència artificial i l'optimització dels prompts enviats als models d'IA. Es van aplicar sistemes de validació i neteja de respostes JSON per garantir estabilitat al fronted i evitar errors durant el processament de les recomanacions de seguretat.

Tecnologies utilitzades

- **Frontend / Framework:**
S'ha utilitzat React.js per fer la llibreria orientada a components, vite per l'entorn de compilació i JavaScript ES6+.
- **Estils i Disseny UI:**
Utilització de Tailwind CSS que es un framework d'estils utilitaris.
- **Backend i Serveis Cloud:**
Firebase SDK, Firebase Authentication i Firebase Realtime Database.
- **Seguretat / Criptografia:**
Web Crypto API (window.crypto), Algorisme AES-256-GCM.
- **Models d'IA:**
Google Gemini API, OpenAI APIs, ChatAnywhere.
- **Control de versions:**
Git i GitHub.

Explicació del codi i Decisions tècniques clau.

El projecte es divideix en un esquema modular: **src/components/** per a la interfícies i **src/utils/** per a la lògica de negoci.

Generació Segura (src/utils/passwrodGenerator.js)

No es va utilitzar l'ús de `Math.random()` perquè és predictable i perillós per a la ciberseguretat. Es va optar per la Web Crypto API, que genera valors aleatoris reals.

```
const array = new Uint32Array(longitud);  
window.crypto.getRandomValues(array);
```

El Xifratge Local GCM (src/utils/encryption.js)

Abans d'enviar qualsevol dada a Firebase, s'aplica un xifratge simètric AES-26-GCM és una decisió tècnica clau perquè, a diferència de models antics com CBC, proporciona autenticitat i integritat, evitant atacs de modificació de dades.

Connexió Segura a Firebase (src/components/HistoryPanel.jsx)

Per emmagatzemar l'historial, les dades es guarden en el node de l'usuari autèntic. Així s'assegura l'aïllament del contingut:

Reptes, aprenentatges i reflexió sobre la IA

Reptes Tècnics Resultats

- **Sincronització Asíncrona**

Treballar amb bases de dades en temps real implica controlar latències i talls de xarxa. Es va resoldre mitjançant una arquitectura defensiva basada en blocs `try/catch` i control d'estat de càrrega a React.

- **Estabilitat del JSON de la IA**

Els models de llenguatge tendeixen a afegir text introductori com ````json. Per solucionar-ho, es va dissenyar un prompt del sistema molt estricte que força la resposta en format pur, acompanyat d'un bloc de neteja de cadenes abans de fer el `JSON.parse()`.

Reflexió sobre l'ús de la IA

El que ha funcionat millor ha estat la velocitat per estructurar components visuals i trobar errors de sintaxi en Java Script, això m'ha estalviat hores de cerca.

Decepcions / Limitacions de la IA:

En l'àmbit de la criptografia, la IA sovint proposava llibreries antigues com CryptoJS, i va ser necessari el meu criteri però forçar l'ús de la Web Crypto API, que és molt més moderna i segura.

Limitacions i millores futures

Les limitacions de millora per transformar l'aplicació en un entorn de producció real són:

Implementació d'Argon2 / PBKDF2

Actualment, la clau d'encryptació del prototip és estàtica per facilitar proves. El següent pas lògic és demanar una Master Password a l'usuari i derivar la clau de xifratge mitjançant salts i hash de llarga durada.

Autenticació Multifactor

Integrar seguretat biomètrica o claus de pas temporals utilitzant WebAuthn o proveïdors cloud avançats.

Protecció de volatilitat del Clipboard

Programar un script que buidi automàticament el porta-retalls del sistema operatiu 15 segons després que l'usuari hagi copiat la contrasenya.

Xifratge de Metadades

Xifrar també camps com la puntuació o les dates de creació de les credencials per evitar que tercers puguin extreure patrons dels usuaris.

Instruccions d'instal·lació i ús

Requisits previs

Disposar de l'entorn d'execució Node.js i el gestor de paquets npm.

Clonar i instal·lar dependències

Utilitzar les comandes:

```
git clone https://github.com/allobera1341-a11y/PasswordManager.git  
cd PasswordManager  
npm install
```

```

C:\Users\llobe>git clone https://github.com/allobera1341-ally/PasswordManager.git
Cloning into 'PasswordManager'...
remote: Enumerating objects: 11491, done.
remote: Counting objects: 100% (11491/11491), done.
remote: Compressing objects: 100% (6724/6724), done.
remote: Total 11491 (delta 4487), reused 11469 (delta 4465), pack-reused 0 (from 0)
Receiving objects: 100% (11491/11491), 38.42 MiB | 7.57 MiB/s, done.
Resolving deltas: 100% (4487/4487), done.
Updating files: 100% (18065/18065), done.

C:\Users\llobe>cd PasswordManager

C:\Users\llobe\PasswordManager>npm install

```

Configurar l'entorn (.env)

Crear un fitxer anomenat .env a l'arrel del projecte i introduir les credencials dels serveis de Firebase i de l'API d'IA utilitzada:

```

# AI Secure Password Manager - Firebase Configuration
VITE_FIREBASE_API_KEY=AIzaSyCTy3jiiAlbbps48fHlmzNYhM-8MLBPNbQ
VITE_FIREBASE_AUTH_DOMAIN=iapasswordmanager.firebaseio.com
VITE_FIREBASE_PROJECT_ID=iapasswordmanager
VITE_FIREBASE_STORAGE_BUCKET=iapasswordmanager.firebaseio.com
VITE_FIREBASE_MESSAGING_SENDER_ID=350717821498
VITE_FIREBASE_APP_ID=1:350717821498:web:e9615545da446db8c92b03
VITE_FIREBASE_DATABASE_URL=https://iapasswordmanager-default-rtdb.europe-west1.firebaseio.com
VITE_FIREBASE_MEASUREMENT_ID=G-NX7675TD6D

# AI Assistant Configuration
VITE_AI_API_KEY=sk-ZsMLtX0DXYvjWgchAzdb2uSIiZ4zUkeKWKU06mQv85jjPIs6
VITE_AI_API_URL=https://api.chatanywhere.tech

```

Executar en entorn de desenvolupament

```

C:\Users\llobe\PasswordManager>npm run dev

> ai-secure-password-manager@0.1.0 dev
> vite

Re-optimizing dependencies because lockfile has changed

VITE v5.4.21 ready in 523 ms

➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help

```

L'aplicació s'aixecarà i estarà accessible des del navegador a l'adreça <http://localhost:5173/>